Programación Web

- Semestre 5 · Clase 9
- Unidad 2 · Luces, mapas y clases

La clase pasada:
materiales, luces y texturas.
Quedó una mesa de billar v0

Hoy la vamos a convertir en
una sala de billar completa:
paredes, techo, luces, sombras,
cuatro patas y un taco

Hoy vemos

Sombras · Mapas · Repeat · Clases

Luces y sombras

Ya conocemos tres luces:
Ambient · Directional · Point

Hoy sumamos dos más, para la lámpara de la mesa y el ambiente

SpotLight: un foco, como una lámpara de techo

```
const lampara = new THREE.SpotLight(0xfff0d0, 140);
lampara.position.set(0, 5.5, 0);
lampara.target.position.set(0, 2.3, 0);
lampara.angle = Math.PI / 3.2;
lampara.penumbra = 0.5;
scene.add(lampara);
scene.add(lampara.target);
```

angle: qué tan abierto es el cono

penumbra: 0 = borde duro, 1 = borde suave

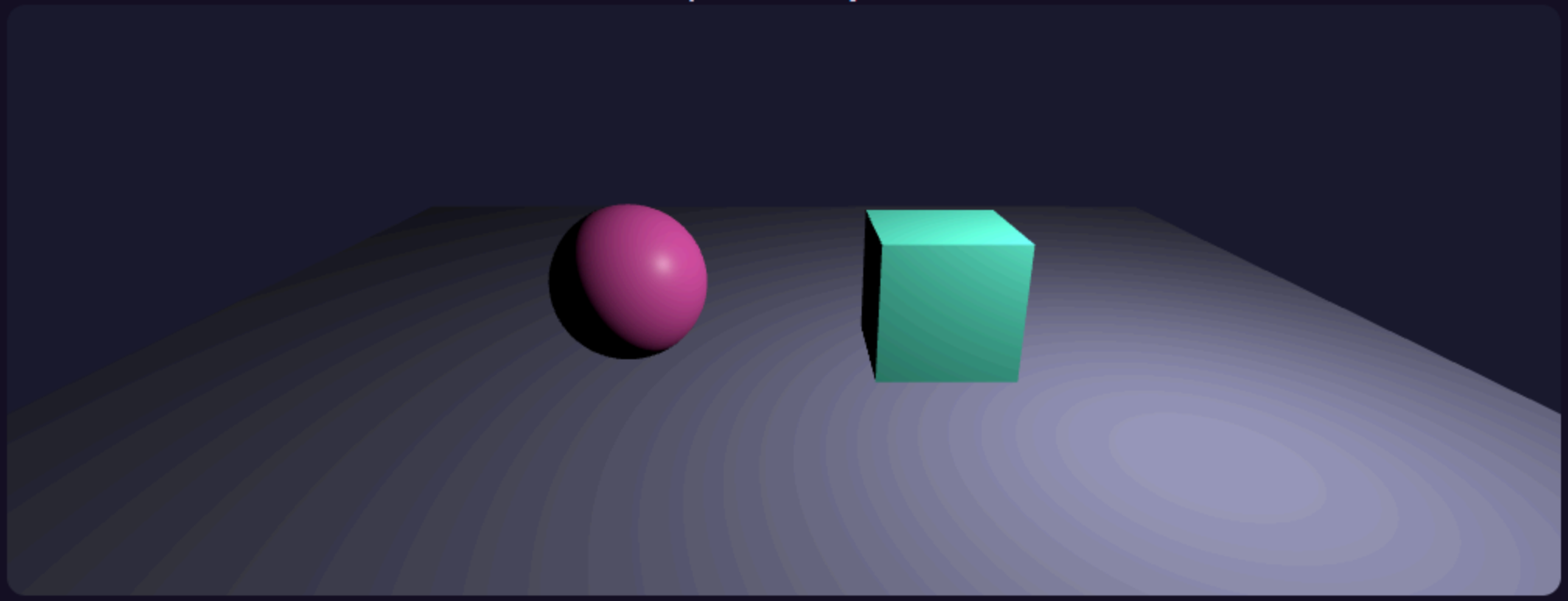
target: hacia dónde apunta

El target también se agrega a la escena

HemisphereLight: luz de cielo y de suelo

```
const cielo = new THREE.HemisphereLight(
  0xbcd4ff,   // color de arriba
  0x5a3a20,   // color de abajo
  0.35        // intensidad
);
scene.add(cielo);
```

Cambien el tipo de luz y la intensidad



luz Point intensidad 1.00

Hasta ahora las bolas flotan
sobre la mesa: no hay sombra
que las pegue al paño

Para que haya sombras hay tres interruptores

1. El renderer las activa
2. La luz las proyecta
3. Los objetos proyectan o reciben

```
renderer.shadowMap.enabled = true;

lampara.castShadow = true;

bola.castShadow = true;        // proyecta sombra
pano.receiveShadow = true;     // recibe sombra
```

castShadow = "yo tapo la luz"

receiveShadow = "sobre mí se ve la sombra"

Un objeto puede hacer las dos cosas:

la mesa tapa la luz del piso y recibe la de la lámpara

¿Cómo hace Three.js una sombra?

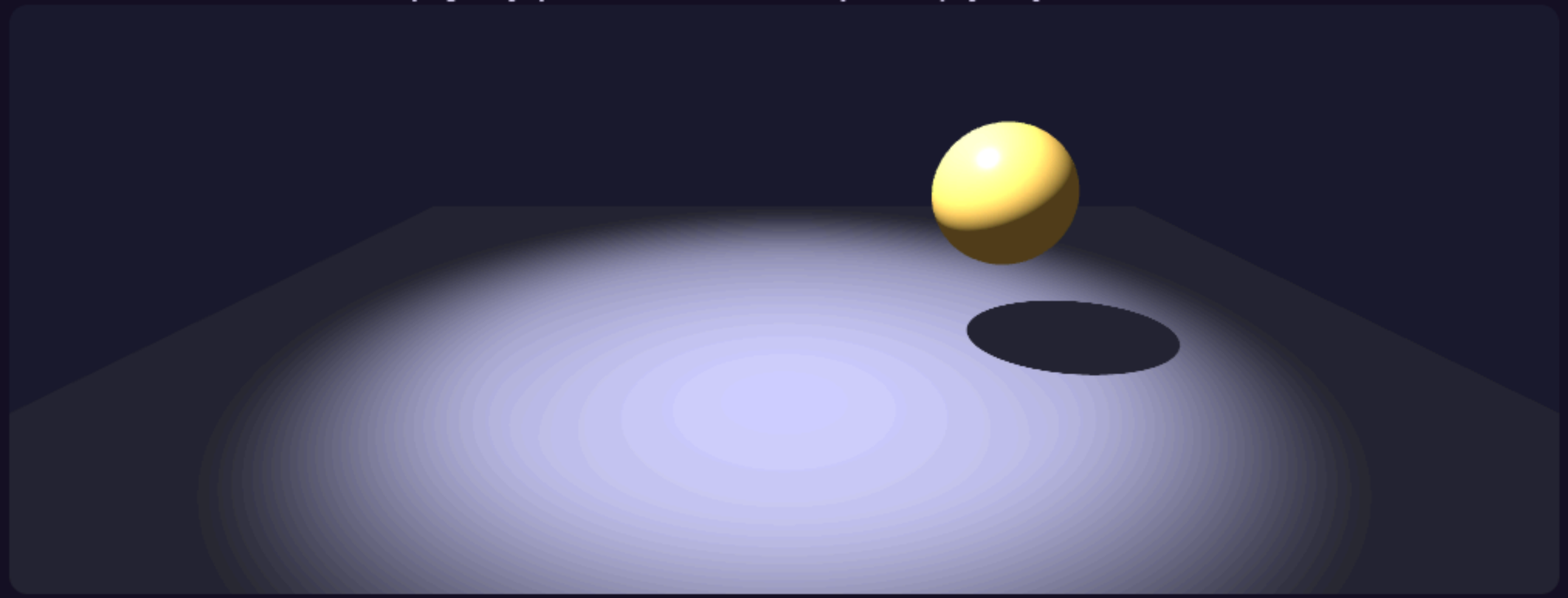
La luz toma una "foto" de la escena
desde donde está ella.
Lo que sale en la foto recibe luz,
lo que queda detrás queda en sombra

Esa foto es el shadow map y tiene una resolución

```
lampara.shadow.mapSize.set(2048, 2048);
```

Poca resolución = sombras pixeladas

Prueben apagar y prender los interruptores, y bajen la resolución



renderer luz bola proyecta piso recibe mapSize 2048

Si ven rayitas raras sobre las superficies, se arregla con el bias

```
lampara.shadow.bias = -0.0005;
```

Un número negativo muy pequeño

Las sombras cuestan
Cada luz con sombra es una "foto" más por cuadro

Regla: sombras solo en la luz
que más se nota (la lámpara)

Ejercicio 1:

Sobre la mesa cuelga una lámpara de salón de billar
que ilumina el paño y deja el resto de la sala en penumbra.

Las bolas deben proyectar su sombra sobre el paño

Mapas

Una textura no tiene que ser solo color

Cada imagen puede controlar
una propiedad distinta del material

map: el color

roughnessMap: qué zonas son más lisas o rugosas

metalnessMap: qué zonas son metal

aoMap: sombras en los huecos

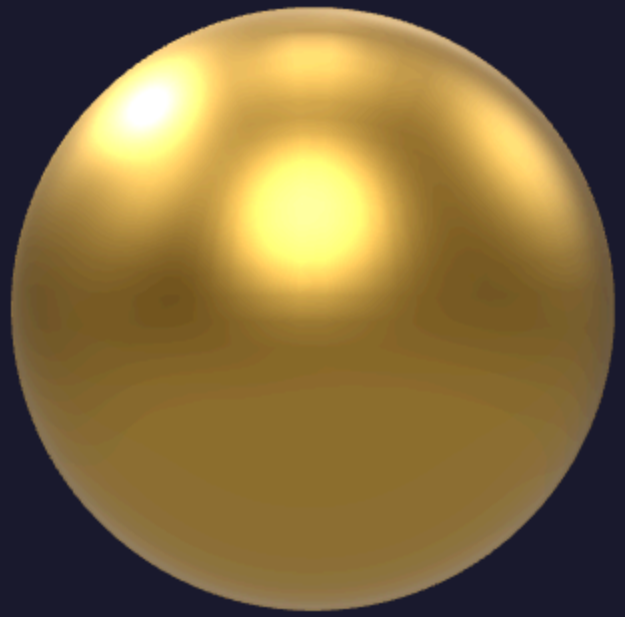
normalMap y bumpMap: relieve falso

displacementMap: relieve real

```
new THREE.MeshStandardMaterial({
  map: colorTextura,
  roughnessMap: rugosidadTextura,
  normalMap: normalTextura,
  aoMap: oclusionTextura
});
```

Cada mapa es una imagen distinta, en escala de grises o de colores raros

Enciendan los mapas uno por uno sobre la misma esfera



☐ map ☐ roughnessMap ☐ normalMap ☐ aoMap ☐ displacementMap ☐ wireframe

bumpMap: escala de grises, claro = alto
Simple, pero poco preciso

normalMap: imagen morada-azulada
Más precisa: guarda hacia dónde mira cada punto

Los dos son relieve falso:
la forma del objeto no cambia, solo cómo le pega la luz

displacementMap sí mueve los vértices

displacementMap necesita muchos vértices para moverlos

```
new THREE.SphereGeometry(1.3, 256, 256);

material.displacementMap = alturaTextura;
material.displacementScale = 0.12;
```

Activen el wireframe en la demo y miren la malla

Ojo con el colorSpace

Solo el mapa de color es sRGB.
Los demás (rugosidad, normal, relieve)
son datos, no colores: no se tocan

```
colorTextura.colorSpace = THREE.SRGBColorSpace;
```

Ya no vamos a repetir el mismo código por cada textura.
Una función hace el trabajo

```
function cargarTextura(archivo, repetirX, repetirY, esColor)
{
  const textura = cargador.load(TEXTURAS + archivo);
  textura.wrapS = THREE.RepeatWrapping;
  textura.wrapT = THREE.RepeatWrapping;
  textura.repeat.set(repetirX, repetirY);
  if (esColor)
  {
    textura.colorSpace = THREE.SRGBColorSpace;
  }
  return textura;
}
```

Y se usa así

```
new THREE.MeshStandardMaterial({
  map: cargarTextura("brick_diffuse.jpg", 5, 2, true),
  bumpMap: cargarTextura("brick_bump.jpg", 5, 2, false),
  roughnessMap: cargarTextura("brick_roughness.jpg", 5, 2, false)
});
```

Todos los mapas de un material llevan el mismo repeat

Para su temática, texturas gratis (licencia CC0)

ambientcg.com

polyhaven.com

Descarguen el set: Color, Roughness, Normal y Displacement

Si usan imágenes suyas,
el archivo no se puede abrir con doble clic

Hay que abrirlo con Live Server

(el navegador bloquea las texturas locales sin servidor)

Ejercicio 2:

La pared del fondo de la sala es de ladrillo.
Debe verse con relieve (que se noten las juntas)
y no brillar como un espejo

Repeat

Si pegan una textura una sola vez
sobre una superficie de 20 metros,
se ve enorme y estirada

Regla:

repeat = medida del objeto

÷ medida de un mosaico

Piso de 20 con tablas de 2: repeat = 10

Una caja larga como el borde de la mesa. Quítenle lo estirado



repeat X 1 repeat Y 1 rotación 0°

Una BoxGeometry pone la textura
entera en cada una de sus 6 caras

El mismo repeat sirve para todas las caras:
un borde largo y uno corto necesitan materiales
distintos

Cada material carga su propia textura

```
const maderaLarga = crearMaterialMadera(4, 1);
const maderaCorta = crearMaterialMadera(2, 1);
```

No sirve clonar la textura antes de que termine de cargar

Para girar la veta de la madera: rotation gira alrededor de center

```
textura.center.set(0.5, 0.5);
textura.rotation = Math.PI / 2;   // 90 grados
```

De lejos, las texturas se ven borrosas cuando se ven en ángulo (el piso)

```
textura.anisotropy = 8;
```

Más anisotropy = más nítido en ángulo, un poco más pesado

Pregunta: la madera del borde se ve aplastada en una cara y bien en otra. ¿Por qué?

Cada cara de la caja mide distinto, pero el repeat es el mismo en todas

Clases

La mesa necesita cuatro patas iguales
y la sala necesita varias bolas

¿Copiamos y pegamos el código una vez por cada pata y cada bola?

Ya sabemos resolverlo: con una función

```
function crearPata(x, z, alto, material)
{
  const malla = new THREE.Mesh(
    new THREE.CylinderGeometry(0.22, 0.16, alto, 24),
    material
  );
  malla.position.set(x, alto / 2, z);
  scene.add(malla);
}

crearPata(-3.8, -1.9, 2, madera);
```

Una clase es una función
que fabrica objetos,
con nombre propio y con memoria

La misma pata, hecha con una clase

```
class Pata
{
  constructor(x, z, alto, material)
  {
    this.malla = new THREE.Mesh(
      new THREE.CylinderGeometry(0.22, 0.16, alto, 24),
      material
    );
    this.malla.position.set(x, alto / 2, z);
    scene.add(this.malla);
  }
}

const pata1 = new Pata(-3.8, -1.9, 2, madera);
```

De función a clase

function crearPata → class Pata

Los parámetros pasan al constructor

El cuerpo de la función es el cuerpo del constructor

const malla → this.malla

crearPata(...) → new Pata(...)

new fabrica un objeto nuevo
y ejecuta el constructor con los datos que le pasan

¿Y si escribo Pata(0, 0, 2, madera) sin el new?
Error: una clase no se puede llamar como función

this significa "este objeto"

Lo que se guarda en this

se puede usar después, desde afuera

```
pata1.malla.position.y = 1;
pata1.malla.material.color.set(0xff0000);
```

Cuatro patas, cuatro new

```
new Pata(-3.8, -1.9, 2, madera);
new Pata( 3.8, -1.9, 2, madera);
new Pata(-3.8,  1.9, 2, madera);
new Pata( 3.8,  1.9, 2, madera);
```

La gran ventaja: si cambian el grosor en la clase, cambian las cuatro patas de un solo golpe

Una clase también puede tener métodos: funciones que viven adentro del objeto

```
class Bola
{
  constructor(color, x, z) { /* ... */ }

  ponerEn(x, z)
  {
    this.malla.position.x = x;
    this.malla.position.z = z;
  }
}

blanca.ponerEn(-2, 0);
```

Convenciones

Nombre de la clase: Mayúscula inicial (Pata)
Nombre del objeto: minúscula inicial (pata1)

Ejercicio 3:

Necesitan varias bolas sobre el paño
(7, 16, las que quieran), de colores distintos.
Cada una brillante, con barniz y con su sombra.

No pueden escribir el código de la bola una vez
por cada bola

Ejercicio 4:

Apoyen un taco cerca de la bola blanca, apuntándole.

Es un solo taco: no necesita clase.
Puede quedar sencillo o verse un poco más artístico:
punta, cinta, detalles

Actividad

Su sala de billar

Una escena 3D de una sala de billar completa con una temática propia

La escena debe tener:

Sala con piso, 4 paredes y techo con texturas
Mesa con bordes y 4 patas (clase Pata)
Bolas (clase Bola) y un taco
Al menos 6 luces, una con sombras
Texturas con repeat bien calculado y al menos 2 mapas además del color

Cada uno con su temática

Salón vaquero · Casino · Espacio Medieval · Neón retro · Bar de playa

Texturas, colores y luces cuentan la historia

Entregan dos cosas:

1. La carpeta del proyecto (index.html, script.js y texturas)
2. Un timelapse de ustedes haciéndola

Condiciones del timelapse

Pantalla completa, no solo la ventana de Visual Studio

Sin cortes ni ediciones: un solo video grabado

Se puede acelerar hasta 4 veces (máximo)

Material de apoyo en Blackboard:

El PDF de esta clase
La guía paso a paso en PDF
Dónde conseguir texturas gratis
Un video de cómo queda

Revisión

Sombras: tres interruptores (renderer, luz, objetos)
y el shadow map (resolución y bias)

Mapas: cada imagen controla una propiedad.
Solo el de color es sRGB

Repeat: medida del objeto ÷ medida del mosaico,
y un material por cada tamaño

Clases: una función que fabrica objetos.

constructor, this, new y métodos

¿Dudas?
¿Algo que no cuadró?

La próxima clase

Ya tenemos la parte estética: la sala se ve bonita

Ahora empezamos a ver cómo se mueve
para poder calcular y jugar:
eventos (mouse y teclado) y luego colisiones

Nos vemos la próxima clase

Si me quieren agregar



Steam: 317510869



LoL: Luxari#Lan

Correo institucional:
jjardila@libertadores.edu.co